FORMA

Invoice Studio | Local Developer Setup & Handover Guide

Purpose: help a developer extract, run, test, and verify the supplied full-stack invoice application on a local machine. This guide describes the project as delivered and calls out steps that require configuration or manual verification.

Project archive: forma-invoice-generator-fullstack.zip

Technology: FastAPI (Python), SQLite, HTML, CSS, vanilla JavaScript, SMTP email integration.

Important: Email delivery cannot work until valid SMTP settings are supplied. The project does not include authentication and should remain local during evaluation.

1. Project contents

Extract the ZIP archive. It contains the following files and directories:

Path	Purpose
app.py	FastAPI application, API routes, SQLite persistence, invoice validation/calculation, SMTP email endpoint.
requirements.txt	Python dependencies, including test dependencies.
.env.example	Template for database and SMTP environment settings. Copy to .env and configure it.
.gitignore	Excludes local secrets, database files, and Python cache files from Git.
README.md	Project overview, endpoints, calculation rules, and known limitations.
static/admin.html	Admin dashboard, invoice list, clients, and settings page.
static/admin.js	Admin page data loading and interactions.
static/invoice.html	Invoice editor, custom fields, line items, live preview, print controls.
static/invoice.js	Editor state, frontend validation, API calls, calculations, save/email/print interactions.
static/styles.css	Shared responsive styling and print-specific rules.
tests/test_api.py	Automated API tests for calculation, validation, CRUD, status updates, and deletion.

2. Requirements

• Python 3.10 or newer.

• A terminal and permission to create a virtual environment and install Python packages.

• A modern browser such as Chrome, Firefox, or Edge.

• Internet access for the first dependency installation.

• Optional: SMTP credentials from an email provider if testing actual invoice email delivery.

3. Extract the archive

Download forma-invoice-generator-fullstack.zip and extract it. Open a terminal in the extracted folder named forma-invoice-generator. Confirm that app.py, requirements.txt, README.md, static/, and tests/ are visible in that folder.

Linux/macOS: you can extract through the file manager, or use the following command if the ZIP is in your Downloads folder:

```
cd ~/Downloads
unzip forma-invoice-generator-fullstack.zip
cd forma-invoice-generator
```

If the ZIP is stored elsewhere, replace ~/Downloads with the actual folder path.

4. Create a Python environment and install dependencies

Linux / macOS

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
pip install -r requirements.txt
```

Windows PowerShell

```
py -m venv .venv
.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install -r requirements.txt
```

If PowerShell blocks activation, use an approved local policy or activate the environment through Command Prompt with .venv\Scripts\activate.bat. Do not bypass a company security policy.

Expected result: the dependencies install without errors. If installation fails, record the full error and Python version using python --version.

5. Configure environment variables

Create a local .env file by copying .env.example. Keep .env private and never commit it to a public repository.

Linux / macOS

```
cp .env.example .env
```

Windows PowerShell

```
Copy-Item .env.example .env
```

Open .env in a text editor. Database configuration can usually remain at its example value:

```
DATABASE_PATH=./data/invoices.db
```

To test email delivery, enter SMTP settings supplied by the project owner or email administrator:

```
SMTP_HOST=smtp.your-provider.example
SMTP_PORT=587
SMTP_USE_SSL=false
SMTP_USER=your-authorized-sender@example.com
SMTP_PASSWORD=your-provider-app-password
SMTP_FROM=your-authorized-sender@example.com
```

These are placeholders, not working credentials. Use values provided by the owner. Port 587 normally uses STARTTLS; port 465 commonly uses implicit SSL with SMTP_USE_SSL=true. Confirm the correct settings with the email provider. Restart the app after editing .env.

Do not send real credentials through source control, commit them to Git, or paste them into public issue trackers. The email endpoint sends a plain-text invoice summary; it does not attach a PDF.

6. Start the application

```
uvicorn app:app --reload
```

Keep the terminal running. On successful startup, Uvicorn reports that it is serving on http://127.0.0.1:8000. If the port is already in use, stop the other process or run on a different port, for example: uvicorn app:app --reload --port 8001.

Open these URLs in your browser:

• **Admin dashboard:** http://127.0.0.1:8000/

• **Invoice editor:** http://127.0.0.1:8000/invoice

• **Health check:** http://127.0.0.1:8000/api/health

The SQLite database is created automatically in the data/ directory on first use, unless DATABASE_PATH has been changed. Stop the server with Ctrl+C.

7. Functional walkthrough

A. Admin dashboard

• Open the admin dashboard. It should show an empty state when no invoices have been saved.

• Use Create invoice to open the invoice editor.

• After saving, return to the dashboard and confirm that invoice totals and invoice counts update.

• Open Invoices to search by invoice number/client and filter by Draft, Sent, Paid, or Overdue.

• Use Open to edit a saved invoice. Use Status to cycle status values.

• Use Delete only after checking the confirmation prompt. Deletion is permanent in this prototype.

• Open Clients to see client records derived from saved invoices.

• Settings stores business defaults in the browser's localStorage; it is not a server-wide settings store.

B. Invoice editor

• Enter business and client information. Client email and required identity/date fields must be valid.

• Add at least one line item. Each item needs a description, quantity greater than zero, and a non-negative unit price.

• Try adding, editing, and removing multiple items.

• Add a custom field, change its label/value, and toggle Show. Hidden fields should not appear in the preview or printed invoice.

• Change discount and tax values and check that the preview and totals update.

• Save the invoice, reload the browser page, and reopen it from the admin page to verify database persistence.

• Use Print / Save PDF and inspect the browser print preview before saving.

C. Required calculation case

Create an invoice with one line item: quantity 2, unit price $50.00, discount $10.00, tax rate 10%. Expected values:

Subtotal	$100.00
Discount	–$10.00
Taxable amount	$90.00
Tax (10%)	$9.00
Grand total	**$99.00**

The backend rounds each line total to cents, sums the rounded lines, applies the fixed discount, calculates tax on the discounted subtotal, rounds tax to cents, and adds tax to the discounted subtotal.

8. Test invalid inputs

• Try a blank invoice number, business name, or client name.

• Try an invalid client email or business email.

• Try a due date earlier than the issue date.

• Try a zero or negative quantity and a negative unit price.

• Try a negative discount or tax rate above 100%.

• Try a discount greater than the subtotal.

• Try saving two invoices with the same invoice number; the API should reject the duplicate.

• Try a custom field with Show disabled and check the preview and print output.

• Try email delivery before configuring SMTP; the app should report that email is not configured instead of claiming delivery succeeded.

9. Run the automated tests

With the virtual environment active and dependencies installed, stop the development server if needed, then run:

```
pytest -q
```

The supplied tests cover the $99 calculation, invoice readback, custom-field persistence, invalid values, due-date order, CRUD updates, status changes, and deletion. Report the actual test output to the project owner. Do not claim tests passed unless they were run on the machine.

10. API endpoints

Method	Endpoint	Purpose
GET	/api/health	Service health check.
GET	/api/invoices?search=&status;=all	List invoices; optional search and status filter.
POST	/api/invoices	Create invoice; returns saved invoice.
GET	/api/invoices/{id}	Retrieve one invoice.
PUT	/api/invoices/{id}	Replace invoice details and line items.
PATCH	/api/invoices/{id}/status	Change status to Draft, Sent, Paid, or Overdue.
DELETE	/api/invoices/{id}	Delete invoice and its line items.
POST	/api/invoices/{id}/email	Send plain-text invoice email to a supplied recipient.

11. Email delivery test

• Confirm SMTP values are valid and authorized by the provider.

• Restart Uvicorn after changing .env.

• Create and save an invoice with a real, valid client email address that you control.

• Enter that same test address in Send invoice to and click Send invoice email.

• Confirm the UI reports success, then check the inbox and spam folder.

• Confirm the invoice status changes to Sent only after successful SMTP delivery.

• If delivery fails, capture the error without sharing credentials. Check hostname, port, SSL mode, authentication, sender authorization, and network restrictions.

The email function does not attach a PDF. It sends a text summary of invoice line items, dates, notes, and totals.

12. Troubleshooting

Symptom	Checks
python / pip not found	Install Python 3.10+ and ensure the executable is on PATH; use python3 on many Linux distributions.
Package installation fails	Check Python version, internet access, and the complete pip error. Activate the intended virtual environment.
ModuleNotFoundError: app	Run Uvicorn from the project root, where app.py exists.
Port 8000 already in use	Stop the other server or run with --port 8001 and open that port in the browser.
Database errors	Check write permissions for the project directory and the DATABASE_PATH parent folder.
Duplicate invoice number	Invoice numbers must be unique. Change the number and save again.
Email configuration error	Set SMTP_HOST and SMTP_FROM, add valid credentials if required, and restart the server.
SMTP delivery fails	Check provider-specific SMTP settings, credentials/app password, TLS mode, sender verification, firewall, and provider restrictions.
Print includes unexpected UI	Use the browser print preview. Only the invoice should be visible; report any browser-specific issue with a screenshot.

13. Security, scope, and known limitations

• This is a local evaluation prototype, not a production-ready accounting platform.

• There is no authentication or authorization. Do not expose it to the public internet.

• SMTP credentials belong only in local .env; do not commit .env or share it in chat.

• SQLite stores the invoices on the local machine. Back up the database if you need to retain test data.

• Client records are derived from invoices; there is no independent client-management table.

• Invoice statuses are manually managed except that a successful email send marks the invoice Sent.

• The email contains a plain-text invoice summary, not a PDF attachment.

• The app does not process payments, create accounting journal entries, or integrate with external accounting services.

• Browser settings are stored in localStorage and are local to that browser profile.

• Before production use, add authentication/authorization, CSRF and abuse protections as appropriate, deployment configuration, backups, logging/monitoring, privacy/security review, and stronger automated coverage.

14. Developer handover checklist

• Project extracted and correct directory opened.

• Python version checked; virtual environment created and activated.

- Dependencies installed successfully.
- .env created from .env.example; no secrets committed.
- Admin dashboard and invoice editor both load locally.
- Create, edit, search, filter, status change, delete, and reload tested.
- Required calculation produces $99.00.
- Negative values, invalid emails, duplicate invoice numbers, and invalid dates rejected.
- Print preview contains only the invoice.
- Automated tests run and output reported accurately.
- SMTP configured and actual email delivery verified, or clearly recorded as untested.
- Known limitations and any issues documented for the owner.

End of guide. If a step fails, send the project owner the step number, exact command, complete error text, operating system, and Python version. Never send SMTP passwords or other secrets.